Data Wrangling with ERA5 Data Files

Or, How I Learned To Stop Worrying And Love The ERA5 File Format

Contents

1. TL;DR **1**
2. Introduction **1**
3. ERA5 Files **1**
4. Tools **5**
5. Environment **5**
6. Get Data **5**
7. Exploration **8**
8. Data Extraction **8**
9. Data Merge **8**
10. Regridding **8**
11. Appendices **13**
A. Installing miniforge 13
B. Tool Options 14
C. CDO Options 17
D. NCO Options 19

1. TL;DR

- ERA5 files contain GRIB2, or NetCDF4 formatted data. These are often in a zipped format without the .zip suffix (still have the .nc suffix). If you have file-type related issues, try unzipping the ERA5 file.
- You will have the most options for data manipulation tooling if you are on a linux or macOS machine or are using WSL2 in Windows. Windows Powershell is fine, but it will limit your tool choices.
- Recommendations: Panoply for visualization and data information. NCO for basic data manipulation and structural changes. CDO for more complicated data manipulation like regriding and merging. Python for complicated processing.
- This document contains learning materials and exercises. If you just want information about the tools and examples of what they can do, please proceed to the appendices.

2. Introduction

This document together with the files in era5_exercises.zip is intended to provide a user with the knowledge and tools to effectively manage ERA5 files.
The exercises can be run on Linux, MacOS, Windows, or Windows WSL (following the Linux specific sections). Exercise differences between the OSes will be pointed out when needed.

Parts of this document include a section on ERA5 files, what they are, where they come from etc. Then a section on creating a Conda environment with all the software needed to view and manipulate the ERA5 files. After that, the next section will give you the tools to download ERA5 files. That is followed by a section on exploring ERA5 data and the tools to use. The next two sections address extracting data from a file and combining data into a file. The last section covers regridding by modifying the grid of an RTMA dataset to the grid of an ERA5 dataset making merging and calculations possible.

3. ERA5 Files

ERA5 is a global atmospheric reanalysis dataset — it combines historical observations with a numerical weather model to produce a physically consistent record of the atmosphere on a regular grid. The name ERA5 stands for ECMWF Reanalysis v5.

These files are a product of the ECMWF - European Centre for Medium-Range Weather Forecasts. The ECMWF runs the IFS NWP in reanalysis mode over the historical record to produce ERA5 data. (IFS = , NWP =)

Most ERA5 files contain data in a NetCDF4, GRIB2, or a .zip format. Extracting the contents of the .zip in an ERA5 file will reveal a NetCDF4, or GRIB2 formatted dataset. Figures 1 and 2.

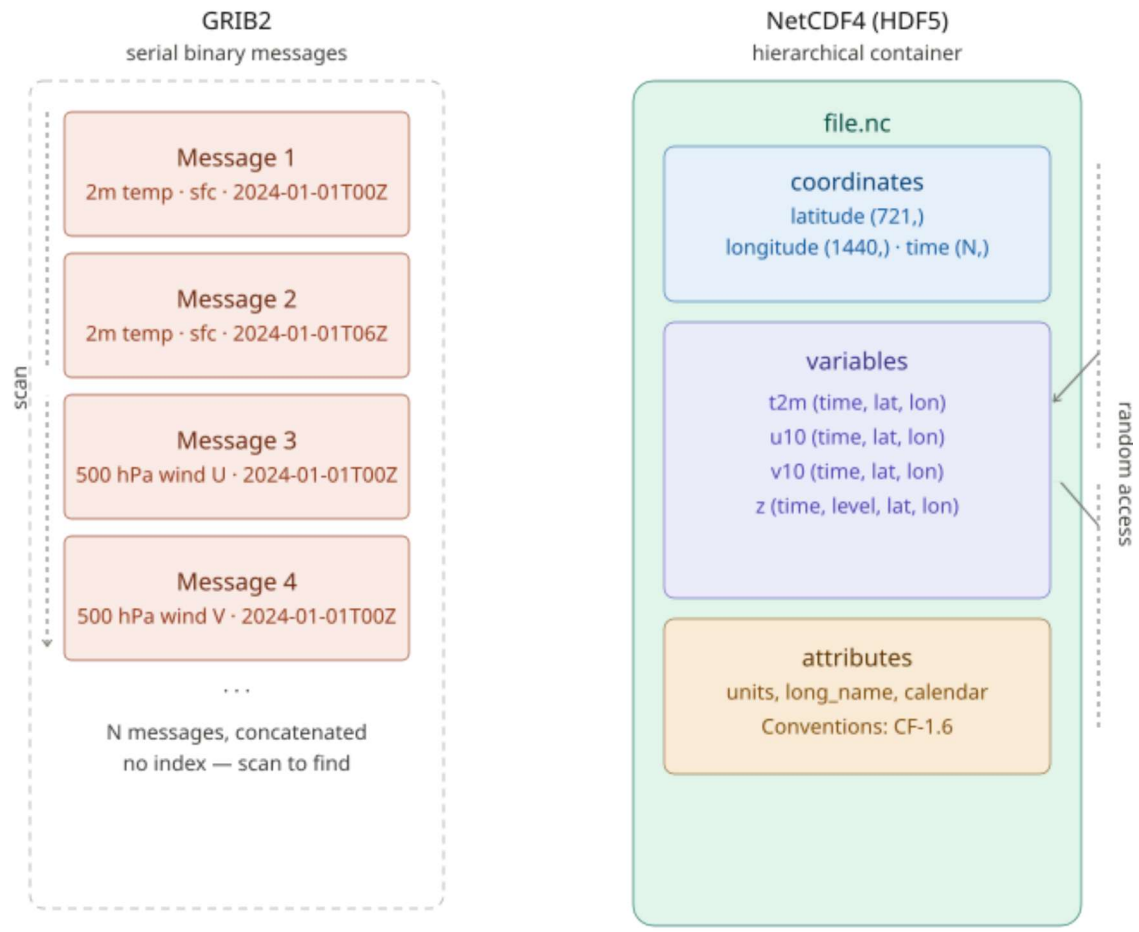

Figure 1. Descriptions of the GRIB2 and NetCDF4 data formats.

Property	GRIB2	NetCDF4
Underlying format	Custom binary	HDF5
Structure	Serial messages	Hierarchical groups
Random access	No — scan serially	Yes — by variable
Metadata location	Per-message header	File-level attributes
Python read tool	cfgrib / eccodes	xarray / netCDF4
file command output	GRIB edition 2	HDF5 data
CDS default?	Yes	Optional (request it)
Industry origin	NWP / operational wx	Scientific / research

Figure 2. Property comparison between GRIB2 and NetCDF4 file formats.

Source	Notes
CDS (Copernicus Climate Data Store)	The primary access point; run by ECMWF under the Copernicus programme. What you use via `cdsapi`.
AWS Open Data	ERA5 is mirrored on S3 (`s3://era5-pds`); Zarr format, useful for cloud-native access
Google Cloud	Also hosts ERA5 as part of the Google Public Datasets program
NCAR/RDA	NCAR's Research Data Archive (ds633.0) hosts ERA5 and is commonly used in the US academic/government community — relevant to your NOAA context
Copernicus Marine/Land services	Derived ERA5-based products, but these are post-processed, not raw ERA5

Figure 3. Places to get ERA5 data.

4. Tools

View .nc file structure	ncdump	ncdump -h file.nc
Dump all data		Ncdump file.nc
List Variables		ncdump -v varname file.nc
Show Coordinates		ncdump -c file.nc

A. View file details:

5. Environment

We will primarily use conda as the package manager for this project. Some of the packages must be downloaded from conda-forge to get the correct versions.
In the project files directory, cd into the 3_environment folder. Select the .yml file matching your OS.
Conda create -f environment_OStype.yml

.nc viewers linux - eog (simple), ncview (complicated, powerful)

6. Get Data

It's fine if you have data that you wish to use for this tutorial. Note that completing this section will leave the user with ERA5 data and corresponding RTMA data to complete the merge process covered later. It will also enable the user to download more ERA5 data in the future.

A. ERA5 data

Proceed to the Copernicus website. Register for an ECMWF account. Copy your api code.

https://cds.climate.copernicus.eu/datasets/reanalysis-era5-single-levels?tab=overview

Create and edit a file with the api code as described below.

```
url: https://cds.climate.copernicus.eu/api/v2
key: YOUR-UID:YOUR-API-KEY
verify: 1
```

On Linux or MacOS::

~/.cdsapirc

chmod 600 ~/.cdsapirc

On Windows::

home\User_Dir\.cdsapirc

get_simple_era5_data.py

```
import cdsapi
import zipfile
import os

c = cdsapi.Client()

c.retrieve(
    "reanalysis-era5-single-levels",
    {
        "product_type": "reanalysis",
        "variable": [
            "2m_temperature",           # instant
            "mean_sea_level_pressure",  # instant
            "total_precipitation",      # accum
            "10m_u_component_of_wind",  # instant
        ],
        "year": "2020",
        "month": "01",
        "day": "01",
        "time": "00:00",
        "format": "netcdf",
    },
    "era5_4vars.nc"
)
```

```
# unzip
if zipfile.is_zipfile("era5_4vars.nc"):
    with zipfile.ZipFile("era5_4vars.nc") as zf:
        print(f"Extracted: {zf.namelist()}")
        zf.extractall(".")
```

Many ERA5 datasets are written in .zip format with no warming or .zip suffix. If Panolpy fails to open an ERA5 dataset, check the file's type.

Linux / MacOS

Command:

```
file <<filename>>
```

Example:

```
file rtma_regridded_00z.nc
```

Return (NetCDF file):

```
rtma_regridded_00z.nc: Hierarchical Data Format (version 5) data
```

Command:

```
file <<filename>>
```

Example:

```
file era5_sample.nc
```

Return (zip file):

```
era5_sample.nc: Zip archive data, made by v2.0 UNIX, extract
using at least v2.0, last modified Mar 31 2026 20:39:50,
uncompressed size 1533644, method=deflate
```

Windows Powershell

Command:

```
$bytes = [System.IO.File]::ReadAllBytes("<<filename>>")[0..3]
```

Example:

```
$bytes =
[System.IO.File]::ReadAllBytes("era5_conus_202006.nc")[0..3]
$bytes
```

Return:

```
Will return text containing 4 values.  Interpretation below:
```

`80 75 3 4` → it's a **ZIP file** — rename to `.zip` and extract

`137 72 68 70` → it's **HDF5/NetCDF4** — Panoply should open this fine

`67 68 70 1` → it's **classic NetCDF3** — Panoply should open this fine

7. Exploration

Data exploration should always be the first step taken when working with unfamiliar data. A preferred tool for exploring and getting information from an ERA5 file, NetCDF4 or GRIB2, is Panoply. It's a Java based program that runs on Linux, MacOS, and Windows 11. Let's give it a try.

A. Download a small amount of ERA5 data. Refer back to section 3 if you didn't complete it previously.
B. Download and install Panoply - https://www.giss.nasa.gov/tools/panoply/
C. Try to start Panoply. If it fails it is likely that your machine needs Java JRE 11 or newer installed. Install it and try to run Panoply again.
D.

8. Data Extraction

9. Data Merge

10. Regridding

ERA5 reanalysis data is distributed on a regular 0.25° latitude/longitude grid (approximately 28 km resolution), a coordinate system and resolution that rarely aligns with the grids used by operational NWP models, ML inference pipelines, or observational datasets. Regridding — the process of interpolating a gridded field from one coordinate system and resolution to another — is therefore a prerequisite for nearly every downstream use of ERA5 data in a production weather modeling workflow.

The primary reasons regridding is required include:

- **Spatial resolution mismatch.** ERA5's native 0.25° grid does not match the input or output grids of most target models. CorrDiff, for example, expects data on a specific coarse grid, and any mismatch in resolution will produce incorrect or failed inference.
- **Projection differences.** ERA5 uses a plate carrée (regular lat/lon) projection, while operational models such as HRRR use Lambert Conformal Conic. Regridding handles both the resolution change and the coordinate reprojection simultaneously.

- **ML training sample construction.** Downscaling models require paired low-resolution and high-resolution samples on exactly the same coordinate system. ERA5 at native resolution serves as the high-resolution target, while a coarsened (regridded) version of the same data provides the synthetic low-resolution input.
- **Multi-source data fusion.** Merging ERA5 with observational or analysis datasets — RTMA surface analyses, MRMS radar QPE, or other reanalyses — requires all sources to share a common grid before variables can be combined or compared.
- **Verification and scoring.** Computing error metrics such as RMSE or spectral scores between ERA5 and model output requires both fields to be on the same grid, typically the coarser of the two, to avoid artificially inflating skill scores through interpolation artifacts.

Convert RTMA data to the same format as ERA5 data.

ERA5 file parms

```
❯ python3 -c "
import xarray as xr, numpy as np

for name, f in [
    ('ERA5 instant',
'era5_raw/era5_20230101_extracted/data_stream-oper_stepType-instant.nc'),
    ('ERA5 accum',
'era5_raw/era5_20230101_extracted/data_stream-oper_stepType-accum.nc'),
]:
    ds = xr.open_dataset(f, engine='netcdf4')
    print(f'\n=== {name} ===')
    print('Dims:  ', dict(ds.dims))
    print('Coords:', list(ds.coords))
    print('Vars:  ', list(ds.data_vars))
    print('Lat:   ', float(ds.latitude.min()), '→', float(ds.latitude.max()))
    print('Lon:   ', float(ds.longitude.min()), '→', float(ds.longitude.max()))
    print('Res:   ', round(float(ds.latitude.diff(\"latitude\").mean()), 4), 'deg')
"

=== ERA5 instant ===
<string>:10: FutureWarning: The return type of `Dataset.dims` will be changed to return a set of
dimension names in future, in order to be more consistent with `DataArray.dims`. To access a
mapping from dimension names to lengths, please use `Dataset.sizes`.
Dims:   {'valid_time': 24, 'latitude': 141, 'longitude': 281}
Coords: ['number', 'valid_time', 'latitude', 'longitude', 'expver']
Vars:   ['t2m', 'd2m', 'u10', 'v10', 'sp', 'msl']
Lat:    20.0 → 55.0
Lon:    -130.0 → -60.0
```

```
Res:    -0.25 deg

=== ERA5 accum ===
<string>:10: FutureWarning: The return type of `Dataset.dims` will be changed to return a set of
dimension names in future, in order to be more consistent with `DataArray.dims`. To access a
mapping from dimension names to lengths, please use `Dataset.sizes`.
Dims:   {'valid_time': 24, 'latitude': 141, 'longitude': 281}
Coords: ['number', 'valid_time', 'latitude', 'longitude', 'expver']
Vars:   ['tp']
Lat:    20.0 → 55.0
Lon:    -130.0 → -60.0
Res:    -0.25 deg
```

RTMA file parms

```
❯ python3 -c "
import cfgrib, numpy as np
ds = cfgrib.open_dataset('rtma_raw/rtma_20230101_00z.grb2_wexp',
    filter_by_keys={'shortName': '2t'}, backend_kwargs={'indexpath': ''})
print('=== RTMA ===')
print('Dims:  ', dict(ds.dims))
print('Coords:', list(ds.coords))
mid = ds.latitude.shape[0] // 2
dlat = float(np.diff(ds.latitude[mid, :]).mean())
dlon = float(np.diff(ds.longitude[mid, :]).mean())
print('Lat range:', float(ds.latitude.min()), '→', float(ds.latitude.max()))
print('Lon range:', float(ds.longitude.min()), '→', float(ds.longitude.max()))
print('Approx spacing (center row): dlat=', round(dlat,4), 'dlon=', round(dlon,4))
"
=== RTMA ===
<string>:6: FutureWarning: The return type of `Dataset.dims` will be changed to return a set of
dimension names in future, in order to be more consistent with `DataArray.dims`. To access a
mapping from dimension names to lengths, please use `Dataset.sizes`.
Dims:   {'y': 1597, 'x': 2345}
Coords: ['time', 'step', 'heightAboveGround', 'latitude', 'longitude', 'valid_time']
Lat range: 19.228975999999996 → 57.08856116148375
Lon range: 221.62680071763828 → 300.95785205842213
Approx spacing (center row): dlat= 0.0005 dlon= 0.0283
```

Single command cdo regrid test check

```
❯ cdo griddes rtma_regridded_00z.nc        # confirm output grid matches target
cdo infon rtma_regridded_00z.nc          # variable list, min/max values

#
# gridID 1
#
gridtype  = lonlat
gridsize  = 39621
xsize     = 281
ysize     = 141
xname     = lon
xlongname = "longitude"
xunits    = "degrees_east"
yname     = lat
ylongname = "latitude"
yunits    = "degrees_north"
xfirst    = 230
xinc      = 0.25
yfirst    = 20
yinc      = 0.25
cdo    griddes: Processed 13 variables [0.20s 89MB]
    -1 :       Date     Time   Level Gridsize    Miss :     Minimum        Mean
Maximum : Parameter name
     1 : 2023-01-01 00:00:00       0    39621    4879 :     -69.000      471.88
3780.5 : orog
     2 : 2023-01-01 00:00:00       0    39621    4879 :      63425.      95866.
1.0252e+05 : sp
     3 : 2023-01-01 00:00:00       2    39621    4879 :      251.82      281.14
301.51 : 2t
     4 : 2023-01-01 00:00:00       2    39621    4879 :      249.04      278.04
298.91 : 2d
     5 : 2023-01-01 00:00:00      10    39621    4879 :     -8.4753     0.98203
17.228 : 10u
     6 : 2023-01-01 00:00:00      10    39621    4879 :     -21.874     0.75838
16.814 : 10v
     7 : 2023-01-01 00:00:00       2    39621    4879 :  0.00057528   0.0070082
0.020488 : 2sh
     8 : 2023-01-01 00:00:00      10    39621    4879 :      0.0000      207.98
359.75 : 10wdir
     9 : 2023-01-01 00:00:00      10    39621    4879 :      0.0000      3.9432
22.102 : 10si
    10 : 2023-01-01 00:00:00      10    39621    4879 :      0.0000      6.1186
29.344 : i10fg
    11 : 2023-01-01 00:00:00       0    39621    4879 :      1.0000      12995.
16000. : vis
    12 : 2023-01-01 00:00:00       0    39621    4879 :      1.0000      8978.8
16000. : ceil
    13 : 2023-01-01 00:00:00       0    39621    4879 :      0.0000      57.477
100.00 : tcc
cdo    infon: Processed 515073 values from 13 variables over 1 timestep [0.21s 90MB]
```

Regrid parms file

```
cat > target_grid.txt << 'EOF'
gridtype  = lonlat
xsize     = 281
ysize     = 141
xfirst    = -130.0
xinc      = 0.25
yfirst    = 20.0
yinc      = 0.25
EOF
```

Single CDO command regrid

```
❯ cdo -f nc4 remapbil,target_grid.txt \
    rtma_raw/rtma_20230101_00z.grb2_wexp \
    rtma_regridded_00z.nc
cdo    remapbil: Bilinear weights from curvilinear (2345x1597) to lonlat (281x141)
grid
cdo    remapbil: Processed 48684545 values from 13 variables over 1 timestep [2.03s
430MB]
```

Check output lat/lon

```
❯ python3 -c "
import xarray as xr
era =
xr.open_dataset('era5_raw/era5_20230101_extracted/data_stream-oper_stepType-instant.nc
')
rtma = xr.open_dataset('rtma_regridded_00z.nc')
print('ERA5 lat:', era.latitude.values[[0,-1]])
print('RTMA lat:', rtma.lat.values[[0,-1]])
print('ERA5 lon:', era.longitude.values[[0,-1]])
print('RTMA lon:', rtma.lon.values[[0,-1]])
"
ERA5 lat: [55. 20.]
RTMA lat: [20. 55.]
ERA5 lon: [-130.  -60.]
RTMA lon: [-130.  -60.]
```

11. Appendices

A. Installing miniforge

```
Invoke-WebRequest -Uri
"https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-Windows-x86_64
.exe" -OutFile "$env:TEMP\Miniforge3.exe"
Start-Process "$env:TEMP\Miniforge3.exe" -Wait
```

B. Tool Options

ERA5 Tool Reference

Tools organized by task and operating system. ✓ = available natively ⚠ = conda-forge only ✗ = not available

1. Access & Download

Tool	Description	macOS	Linux	Windows (native)	Notes / gotchas
cdsapi	**Python CDS API client — download ERA5 via script**	✓ Available pip install cdsapi or conda install -c conda-forge cdsapi	✓ Available pip install cdsapi or conda install -c conda-forge cdsapi	✓ Available conda install -c conda-forge cdsapi (Miniforge prompt)	Place credentials in ~/.cdsapirc (Linux/macOS) or C:\Users\<user>\.cdsapirc (Windows)
CDS web UI	**Browser-based download from climate.copernicus.eu**	✓ Available No install — any browser	✓ Available No install — any browser	✓ Available No install — any browser	Delivers ZIP archive containing GRIB2 or NetCDF — extract before use

2. Inspect & Validate

Tool	Description	macOS	Linux	Windows (native)	Notes / gotchas
ncdump	**Quick header/content dump for NetCDF files**	✓ Available brew install netcdf	✓ Available conda install -c conda-forge netcdf4 or module load netcdf	✓ Available conda install -c conda-forge netcdf4	ncdump -h file.nc shows dimensions, variables, attributes
grib_ls / grib_dump	**Inspect GRIB2 contents and metadata (ecCodes CLI)**	✓ Available brew install eccodes	✓ Available sudo apt install libeccodes-tools or conda install -c conda-forge eccodes	⚠ Conda-forge (unreliable) conda install -c conda-forge eccodes (no native installer)	grib_ls -n parameter file.grib2 lists all messages with key parameters
wgrib2	**Lightweight GRIB2 inventory and subsetting CLI**	✓ Available brew install wgrib2	✓ Available conda install -c conda-forge wgrib2 or module load wgrib2	⚠ Conda-forge (unreliable) conda install -c conda-forge wgrib2	wgrib2 -s file.grib2 gives quick inventory of all records
xarray	**Open, inspect, and validate ERA5 files in Python**	✓ Available conda install -c conda-forge xarray netcdf4	✓ Available conda install -c conda-forge xarray netcdf4	✓ Available conda install -c conda-forge xarray netcdf4	ds.info(), ds.dims, ds.coords — primary Python inspection interface
cfgrib	**Open GRIB2 files as xarray datasets**	✓ Available conda install -c conda-forge cfgrib eccodes	✓ Available conda install -c conda-forge cfgrib eccodes	⚠ Conda-forge (unreliable) conda install -c conda-forge cfgrib eccodes (pip fails on Windows)	Use filter_by_keys={'shortName': 't'} for multi-variable GRIB files

3. Manipulate & Transform

Tool	Description	macOS	Linux	Windows (native)	Notes / gotchas
CDO	**Climate Data Operators — temporal/spatial ops, format conversion**	✓ Available brew install cdo or conda install -c conda-forge cdo	✓ Available conda install -c conda-forge cdo or module load cdo	⚠ Conda-forge (unreliable) conda install -c conda-forge cdo (no native Windows build)	cdo selvar,t,u,v in.nc out.nc — spatial subsetting, remapping, temporal averaging. WSL2 is the most reliable path on Windows.
NCO	**NetCDF Operators — ncks, ncrcat, ncwa for slicing/merging**	✓ Available brew install nco or conda install -c conda-forge nco	✓ Available conda install -c conda-forge nco or module load nco	⚠ Conda-forge conda install -c conda-forge nco	ncks -v t,u,v in.nc out.nc — best CDO substitute on Windows if conda-forge works; otherwise use WSL2.
xarray + xESMF	**Python regridding: bilinear, conservative, nearest-neighbor**	✓ Available conda install -c conda-forge xesmf	✓ Available conda install -c conda-forge xesmf	⚠ Conda-forge (unreliable) conda install -c conda-forge xesmf (MPI deps may be missing)	Use periodic=True for global ERA5 grids to avoid prime-meridian seam artifacts
xarray	**Subsetting, slicing, unit conversion, masking in Python**	✓ Available conda install -c conda-forge xarray	✓ Available conda install -c conda-forge xarray	✓ Available conda install -c conda-forge xarray	ds.sel(isobaricInhPa=850) — ERA5 lats are descending (90 → -90)

4. Visualization

Tool	Description	macOS	Linux	Windows (native)	Notes / gotchas
Panoply	**GUI viewer for NetCDF and GRIB — quick map plots, no code**	✓ Available giss.nasa.gov/tools/panoply (native app)	✓ Available giss.nasa.gov/tools/panoply (requires Java)	✓ Available giss.nasa.gov/tools/panoply (native installer)	Best no-code option across all platforms for spatial sanity checks
ncview	**Lightweight X11 GUI for rapid NetCDF visualization**	✓ Available brew install ncview (requires XQuartz)	✓ Available sudo apt install ncview or conda install -c conda-forge ncview	✗ Not available Not available natively. Use Panoply or WSL2 + VcXsrv	On remote Linux: ssh -X then ncview — or save PNG with matplotlib
matplotlib + Cartopy	**Python map plots with projections, coastlines, colorbars**	✓ Available conda install -c conda-forge cartopy matplotlib	✓ Available conda install -c conda-forge cartopy matplotlib (headless: use Agg backend)	⚠ Conda-forge (unreliable) conda install -c conda-forge cartopy matplotlib (pip fails)	Headless Linux: matplotlib.use('Agg') before importing pyplot

Tool	Description	macOS	Linux	Windows (native)	Notes / gotchas
QGIS	**Full GIS app — opens NetCDF time-steps, raster layers**	✓ Available `qgis.org installer`	✓ Available `sudo apt install qgis` or `flatpak install qgis`	✓ Available `qgis.org installer`	Good for spatial sanity checks; not for scripted batch workflows

5. Python / ML Pipeline

Tool	Description	macOS	Linux	Windows (native)	Notes / gotchas
MetPy	**CF-convention metadata, unit handling, derived variables**	✓ Available `conda install -c conda-forge metpy`	✓ Available `conda install -c conda-forge metpy`	✓ Available `conda install -c conda-forge metpy`	metpy.parse_cf(ds) adds CF-compliant units and coord awareness to xarray
Dask	**Lazy/parallel loading for large multi-file ERA5 datasets**	✓ Available `conda install -c conda-forge dask`	✓ Available `conda install -c conda-forge dask`	✓ Available `conda install -c conda-forge dask`	xr.open_mfdataset(files, chunks={'time':24}) — essential for multi-month ERA5
rioxarray	**GeoTIFF export, CRS handling, raster I/O on top of xarray**	✓ Available `conda install -c conda-forge rioxarray`	✓ Available `conda install -c conda-forge rioxarray`	✓ Available `conda install -c conda-forge rioxarray`	Use for exporting ERA5 variables as GeoTIFFs for GIS or ML raster pipelines
netCDF4 / h5netcdf	**Low-level NetCDF4 / HDF5 read-write backends for xarray**	✓ Available `conda install -c conda-forge netcdf4 h5netcdf`	✓ Available `conda install -c conda-forge netcdf4 h5netcdf`	✓ Available `conda install -c conda-forge netcdf4 h5netcdf`	CDS now delivers ZIP-wrapped NetCDF4 (HDF5 format) — h5netcdf handles it cleanly

C. CDO Options

CDO — Climate Data Operators

Task reference for ERA5 datasets. CDO chains operators right-to-left; data flows from right to left through the pipeline.

Command Reference

Category	Task	Example

Inspection	**List file contents**	Show all variables, levels, dates in the file
		`cdo sinfo input.nc`
		
		Short summary of dimensions and variables
		`cdo info input.nc | head -20`

	Show variable names	`cdo showvar input.nc`
	Show available levels	`cdo showlevel input.nc`
	Show time steps	`cdo showtimestamp input.nc`
	Show grid info	`cdo griddes input.nc`
Extraction	**Extract one variable**	`cdo selvar,t input.nc output.nc`
	Extract multiple variables	`cdo selvar,t,u,v input.nc output.nc`

	Extract pressure level	Single level (hPa)
		`cdo sellevel,850 input.nc output.nc`
		
		Multiple levels
		`cdo sellevel,500,700,850 input.nc output.nc`

	Extract variable at level (chained)	CDO chains right-to-left — data flows right to left `cdo selvar,t -sellevel,500,850 input.nc output.nc`
	Extract spatial bounding box	Format: lon1,lon2,lat1,lat2 (ERA5 lons are 0-360) `cdo sellonlatbox,230,300,20,55 input.nc conus.nc`
Temporal	**Subset by date range**	`cdo seldate,2020-01-01,2020-12-31 input.nc output.nc`
	Select specific years	`cdo selyear,2018,2019,2020 input.nc output.nc`

	Select specific months	All Januaries across all years
		`cdo selmon,1 input.nc output.nc`
		
		Summer months (JJA)
		`cdo selmon,6,7,8 input.nc output.nc`

	Merge files along time	`cdo mergetime era5_2018.nc era5_2019.nc merged.nc`

Statistics	**Time mean**	Mean over all time steps
		`cdo timmean input.nc output.nc`
		
		Monthly means
		`cdo monmean input.nc output.nc`
		
		Annual means
		`cdo yearmean input.nc output.nc`
		
		Daily means
		`cdo daymean input.nc output.nc`

Category	Task	Example
	Climatology (multi-year mean)	Monthly climatology across all years `cdo ymonmean input.nc climatology.nc`
	Standard deviation	`cdo timstd input.nc output.nc`
	Anomalies from climatology	Subtract monthly climatology from each time step `cdo ymonsub input.nc climatology.nc anomalies.nc`
Regridding	**Bilinear regrid to target grid**	target_grid.txt defines output resolution/projection `cdo remapbil,target_grid.txt input.nc output.nc`
	Conservative regrid	Use for fluxes, precipitation — preserves area integrals `cdo remapcon,target_grid.txt input.nc output.nc`
	Nearest-neighbor regrid	Use for categorical fields (land mask, soil type) `cdo remapnn,target_grid.txt input.nc output.nc`
	Regrid + flip latitude	ERA5 latitudes are descending; some targets expect ascending `cdo remapbil,target_grid.txt -invertlat input.nc output.nc`
Format / Convert	**Convert GRIB2 to NetCDF**	`cdo -f nc4 copy input.grib2 output.nc`
	Convert NetCDF to GRIB2	`cdo -f grb2 copy input.nc output.grib2`
	Set missing value	`cdo setmissval,-9999 input.nc output.nc`
	Extract point value as text	Nearest-neighbor point extract to stdout `cdo outputtab,date,lon,lat,value -remapnn,lon=-97.5/lat=38.5 input.nc`

Chaining	**Chain multiple operators**	CDO chains right-to-left (data flows right to left)
		
		Extract t at 850 hPa, then compute monthly mean
		`cdo monmean -sellevel,850 -selvar,t input.nc output.nc`
		
		Regrid after temporal mean (avoids repeated regridding)
		`cdo remapbil,grid.txt -timmean input.nc output.nc`

Note: file operations that modify metadata (ncrename, ncatted) edit in-place unless an output file is specified.

D. NCO Options

NCO — NetCDF Operators

Task reference for ERA5 datasets. NCO tools: ncks (extract/inspect), ncrcat (concatenate), ncrename, ncatted, ncap2 (arithmetic), ncra (average), ncwa (weighted average), ncbo (binary ops).

Command Reference

Category	Task	Example

Inspection	**Inspect file header**	Equivalent to ncdump -h — shows dims, vars, attributes
		`ncks -M file.nc`
		
		Show metadata only (no data)
		`ncdump -h file.nc`

	Print variable data	`ncks -v t2m file.nc`
	List global attributes	`ncks -M -m file.nc`
Extraction	**Extract one variable**	`ncks -v t2m input.nc output.nc`
	Extract multiple variables	`ncks -v t2m,u10,v10 input.nc output.nc`
	Extract pressure level by value	`ncks -v t -d level,500.0 input.nc output.nc`
	Extract pressure level by index	0-based index — check order with ncdump -h first `ncks -v t -d level,2 input.nc output.nc`
	Temporal subset by date	`ncks -d time,"2020-01-01","2020-12-31" input.nc output.nc`
	Spatial bounding box	Subset lon/lat by value range `ncks -d longitude,230.0,300.0 -d latitude,20.0,55.0 input.nc conus.nc`

Concatenation	**Concatenate along time**	Stack monthly files into one — most common ERA5 use case
		`ncrcat era5_jan.nc era5_feb.nc era5_mar.nc merged.nc`
		
		Wildcard — concatenate all files matching pattern
		`ncrcat era5_2020_*.nc era5_2020_full.nc`

	Concatenate along record dimension	`ncrcat -d time,0,11 input.nc output.nc`
	Append variables from two files	Both files must have same dimensions `ncks -A source.nc destination.nc`
Rename / Metadata	**Rename a variable**	Edits the file in-place (no output file arg needed) `ncrename -v temperature,t input.nc`
	Rename a dimension	`ncrename -d lev,level input.nc`

	Change or add an attribute	Format: -a att_name,var,mode,type,value
		`ncatted -a units,t2m,o,c,"K" input.nc`
		
		Add a global attribute
		`ncatted -a history,global,o,c,"processed ERA5" input.nc`

	Change fill value	`ncatted -a _FillValue,t2m,o,f,-9999.0 input.nc output.nc`
	Delete an attribute	Mode 'd' = delete `ncatted -a long_name,t2m,d,, input.nc`

Category	Task	Example

Arithmetic	**Compute a new variable**	Convert Kelvin to Celsius
		`ncap2 -s "t2m_C=t2m-273.15" input.nc output.nc`
		
		Compute wind speed from components
		`ncap2 -s "wspd=sqrt(u10*u10+v10*v10)" input.nc output.nc`

	Multiply variable by scalar	Convert geopotential (m2/s2) to geopotential height (m) `ncap2 -s "z_m=z/9.80665" input.nc output.nc`
	Subtract two files	File-wise subtraction — e.g. anomaly from climatology `ncbo --op_typ=- field.nc climatology.nc anomaly.nc`

Statistics	**Time average**	Average over all time steps
		`ncra input.nc output.nc`
		
		Average over a subset of files
		`ncra era5_2018.nc era5_2019.nc era5_2020.nc mean.nc`

	Spatial average	Average over all lat/lon — removes spatial dimensions `ncwa -a latitude,longitude input.nc output.nc`
	Weighted spatial average	Weight by cos(lat) to account for grid area distortion `ncap2 -s "wgt=cos(latitude*3.14159/180)" input.nc wgt.nc ncwa -w wgt -a latitude,longitude wgt.nc output.nc`

Note: file operations that modify metadata (ncrename, ncatted) edit in-place unless an output file is specified.